プログラミング演習Ⅲ　課題C1（例題、基本問題、発展問題）

【例題１】 ディレクトリの作成

以下のディレクトリ構造を作成せよ。

```
~/ ┬
   ├ Prog1/
   │
   └ Prog3/ ┬
            └ C1/ ┬
                  ├ basics/
                  ├ test/
                  ├ make_test/
                  │
                  └ array_test/
```

【例題２】 コンパイルと実行（ ~/Prog3/C1/basics/ ）

以下のプログラムを実行し、その出力結果を解答せよ。

hello.c

```
1:    #include <stdio.h>
2:
3:    int main(void) {
4:        printf("Hello World!¥n");
5:        return 0;
6:    }
```

【コンパイル等】

- `gcc hello.c -o myhello` （`hello.c` をコンパイルし，実行ファイル `myhello` を作成）

【実行】

- `./myhello`

  (1)

【例題３】 コマンドライン引数（ ~/Prog3/C1/basics/ ）

以下のソースファイルを作成、コンパイルし、さらに指定された通りに実行し、端末に表示される内容を解答せよ。

repeat.c

```
1:    #include <stdio.h>
2:
3:    int main(int argc, char **argv){
4:       int i;
5:
6:       for(i=0; i<argc; i++){
7:          printf("[%d] %s ", i, argv[i]);
```

```
    }
    printf("¥n");

    return 0;
}
```

【コンパイル】

- gcc repeat.c -o myrepeat

【実行】

- ./myrepeat Hello World!

  (1)

- ./myrepeat A B C

  (2)

【例題４】 整数を引数とする関数、配列を引数とする関数（ ~/Prog3/C1/basics/ ）

以下のソースファイルを作成、コンパイル、実行し、端末に表示される内容を解答せよ。

functions.c

```
#include <stdio.h>
int square(int); //  整数を引数として受け取り、その２乗を出力
int sum(int *, int); //  配列の先頭アドレスと、配列の長さを受け取り、配列内の整数の２乗和を出力

int main(void){
    int n = 10;
    int A[10] = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};

    printf("%d¥n", square(n));
    printf("%d¥n", sum(A, n));

    return 0;
}

int square(int a){
    return a * a;
}

int sum(int *X, int n){
    int i, s = 0;

    for(i = 0; i < n; i++){
        s = s + X[i]*X[i];
    }

```

```
26:     return s;
27: }
```

【コンパイル等】

- `gcc functions.c -o myfunctions`

【実行】

- `./myfunctions`

```
(1)
(2)
```

【例題５】 複数の返り値を持つ関数（ ~/Prog3/C1/basics/ ）

以下のソースファイルを作成、コンパイル、実行した結果を示せ。

substitute.c

```
1:    #include <stdio.h>
2:
3:    void substitute(int *a, int *b) { // int 型のアドレスを２つ受け取る
4:        *a = 10;   //  １つ目のアドレスに対応する変数の値を書き換えている
5:        *b = 100; //  ２つ目のアドレスに対応する変数の値を書き換えている
6:
7:        return;
8:    }
9:
10:   int main() {
11:       int i = 1, j = 2;
12:
13:       printf("i=%d j=%d¥n", i, j);
14:       substitute(&i, &j); // substitute 関数内で行われた処理が、main 関数に反映されている
15:       printf("i=%d j=%d¥n", i, j);
16:
17:       return 0;
18:   }
```

【コンパイル】

- `gcc substitute.c -o mysubstitute`

【実行】

- `./mysubstitute`

```
(1)
(2)
```

【例題６】 malloc による１次元配列の動的確保（ ~/Prog3/C1/basics/ ）

以下のプログラムの空欄について

(1)`malloc` 関数を用いて、長さ n の int 型の配列を確保し，先頭アドレスを int*型にキャストして返す

と動作するように正しく埋め、WebClass には空欄部分に埋めた内容を解答せよ。実行結果が正しいことも確認すること。

malloc1.c

```
#include <stdio.h>
#include <stdlib.h> // atoi 関数を仕様するため

int main(int argc,   char **argv) {
    int i, n, *A;

    if (argc != 2) {     //  引数の個数のエラーを検知する
        fprintf(stderr, "Please input an integer.¥n");
        exit(1);
    }

    n = atoi(argv[1]);   // argv[1]を数字に変換
    A = [                    (1)                    ]
    if(A == NULL){   //メモリ確保に失敗した場合は強制終了
          fprintf(stderr,   “malloc error¥n”);
          exit(1); //  異常終了を表す
     }

    for (i = 0; i < n; i++) A[i] = i * i;
    for (i = 0; i < n; i++) printf("A[%d] = %d¥n", i, A[i]);

    free(A); //確保したメモリを解放する
    return 0;
}
```

【コンパイル等】

- `gcc malloc1.c -o mymalloc1`

【実行】

- `./mymalloc1 10`

```
A[0] = 0
A[1] = 1
      ⋮
A[9] = 81
```

【例題７】 整数列を表すファイルの読み込み（ ~/Prog3/C1/basics/ ）

以下のプログラムの空欄を

(1) `read_array` 関数を用いて、１つ目の引数に示した名前のファイルから整数を読み込み， 長さ n の配列に格納し、その先頭アドレスを `TEST` に返す。

(2) `malloc` を用いて，長さ n の `int` 型の配列を確保し，先頭アドレスを `int*`型として返す

と動作するように正しく埋めよ。さらに、array.txt を作成して、myread_array を実行し、

(3)プログラムの出力結果

を得よ。WebClass で、(1)と(2)の空欄部分に埋めた内容、および(3)の結果として得られる array.txt 内の整数の総和を解答せよ。

read_array.c

```
1:  #include <limits.h>
2:  #include <stdio.h>
3:  #include <stdlib.h>
4:
5:  int count(char *); //  読み込んだファイルに含まれる整数の個数を数える関数
6:  int *read_array(char *,  int); //  ファイル内の整数を読み、配列に格納する関数
7:  void show_array(int *,  int); // int 型の配列の中身を表示する関数
8:
9:  int main(int argc,  char **argv) {
10:     int i, n, *TEST, sum = 0;
11:
12:     n = count(argv[1]); // 1 つ目の引数に示した名前のファイル内の整数の個数を調べる
13:     TEST = [          (1)          ]
14:
15:     show_array(TEST, n);
16:
17:     for (i = 0; i < n; i++)
18:         sum = sum + TEST[i];
19:     printf("%d¥n", sum); //  整数の総和を出力する
20:
21:     free(TEST);
22:     return 0;
23: }
24:
25: int count(char *filename) { //読み込んだファイルに含まれる整数の個数を数える関数
26:     int a, k;
27:     FILE *fp;
28:
29:     fp = fopen(filename,  "r");
30:     if (fp == NULL) {
31:         fprintf(stderr,  "open error %s¥n",  filename);
32:         exit(1);
33:     }
34:
35:     k = 0;
36:     while (fscanf(fp, "%d", &a) != EOF) {
37:         k++;
```

```
38:     }
39:
40:     fclose(fp);
41:     return k;
42: }
43:
44: int *read_array(char *filename, int n) { //  ファイル内の整数を読み、配列に格納する関数
45:     int i, *A;
46:     FILE *fp;
47:
48:     fp = fopen(filename, "r");
49:     if (fp == NULL) {
50:         fprintf(stderr,    "open error %s¥n", filename);
51:         exit(1);
52:     }
53:
54:     A = [                    (2)                    ]
55:
56:     for (i = 0; i < n; i++) {
57:         if (fscanf(fp, "%d", &A[i]) == EOF) { // fp が指すファイル内の整数を読み込む
58:             break;
59:         }
60:     }
61:
62:     fclose(fp);
63:     return A;
64: }
65:
66: void show_array(int *A,    int cnt) {
67:     int i;
68:
69:     for (i = 0; i < cnt; i++) {
70:         if (A[i] < INT_MAX)
71:             printf("%d ",    A[i]);
72:         else
73:             printf("N ");
74:     }
75:     printf("¥n");
76:
77:     return;
78: }
```

array.txt

```
123
456
789
1011
1213
1415
1617
1819
```

【コンパイル】

- `gcc read_array.c -o myread_array`

【実行】

- `./myread_array array.txt`

(3)

【基本問題１】 分割前のプログラム（ ~/Prog3/C1/test/ ）

以下のプログラムを作成、実行し，入力として 27 を与えたときの出力結果を示せ．出力結果は３行分となるので、それぞれを(1)，(2)，(3)に解答すること。

test.c

```
#include <stdio.h>

void input(int *); //  数値入力用の関数
void output(int, int); //  計算結果の出力用関数
int square(int); //  整数を二乗する関数
int cube(int); //  整数を三乗する関数

int main() {
    int n;

    input(&n); //入力した値を n に代入する
    output(square(n), cube(n)); //  ２乗と３乗を出力する

    return 0;
}

void input(int *a) { //数値入力用の関数
    printf("Input an integer: ");
    scanf("%d", a);

    return;
}
```

```
23:
24:  void output(int a, int b) { //  ２つの値を出力する関数
25:      printf("The square is %d.¥n", a);
26:      printf("The cube is %d.¥n", b);
27:  }
28:
29:  int square(int a) { //２乗を計算する関数
30:      return a * a;
31:  }
32:
33:  int cube(int a) { //３乗を計算する関数
34:      return a * a * a;
35:  }
```

【コンパイル】

- gcc test.c -o mytest

【実行】

- ./mytest

(1)
(2)
(3)

【基本問題２】 分割コンパイル（ ~/Prog3/C1/make_test/ ）

新しいディレクトリを作成せよ。さらにそのディレクトリの中に以下のファイルを作成した後、各コマンド(1)，(2)，(3)，(4)を実行して作られるファイルを答えよ。

test.h

```
1:    #include <stdio.h>
2:
3:    void input(int*);
4:    void output(int, int);
5:    int square(int);
6:    int cube(int);
```

main.c

```
1:    #include "test.h"
2:
3:    int main() {
4:        int n;
5:
6:        input(&n);
7:        output(square(n), cube(n));
```

```
8:
9:      return 0;
10: }
```

interface.c

```
1:    #include "test.h"
2:
3:    void input(int *a) {
4:        int z;
5:
6:        printf("Input an integer: ");
7:        z = scanf("%d", a);
8:
9:        return;
10:   }
11:
12:   void output(int a, int b) {
13:       printf("The square is %d. ", a);
14:       printf("The cube is %d.¥n", b);
15:   }
```

arithmetic.c

```
1:    #include "test.h"
2:
3:    int square(int a) {
4:        return a * a;
5:    }
6:
7:    int cube(int a) {
8:        return a * a * a;
9:    }
```

【コンパイル等】

- gcc -c main.c　(1)
- gcc -c interface.c　(2)
- gcc -c arithmetic.c　(3)
- gcc main.o interface.o arithmetic.o -o mytest (4)

【実行】

- ./test

【後処理】

- rm -f main.o interface.o arithmetic.o mytest（次の課題のため，作成した４つのファイルを消去）

【基本問題３】 make を用いた分割コンパイル（ ~/Prog3/C1/make_test/ ）

基本問題２と同じディレクトリで、以下の makefile を作成し，make コマンドを実行し、その結果を示せ．出力結果は４行分となるので、それぞれ(1)，(2)，(3)，(4)に解答すること。

※5、7、9、11、13 行目の先頭はタブ文字によるスペースであることに注意すること

makefile

```
1:  OBJS = main.o interface.o arithmetic.o
2:  CC = gcc
3:
4:  mytest: $(OBJS)
5:          $(CC) $(OBJS) -o mytest
6:  main.o: main.c test.h
7:          $(CC) -c main.c
8:  interface.o: interface.c test.h
9:          $(CC) -c interface.c
10: arithmetic.o: arithmetic.c test.h
11:         $(CC) -c arithmetic.c
12: clean:
13:         rm -f $(OBJS) mytest
```

【コンパイル】

- make

(1)

(2)

(3)

(4)

【コンパイル】（結果が４行にならなかった場合、既にファイルが作成されている可能性がある）

- make clean（clean をターゲットにして mytest、main.o、interface.o、arithmetic.o を削除）
- make （mytest、main.o、interface.o、arithmetic.o がないため、全てが作成される）

【基本問題４】 make の動作（ ~/Prog3/C1/make_test/ ）

基本問題３で作成したファイルについて、make コマンドを使って以下の作業を実現したいときに、端末に入力、実行する内容を答えよ．

(1) main.c，interface.c arithmetic.c をコンパイルして、main.o，interface.o arithmetic.o を作成し、さらにリンクして実行ファイル mytest を作成する。

(2) ディレクトリにある main.o，interface.o arithmetic.o mytest を全て削除する。

(3) interface.c のみをコンパイルして、interface.o を作成する。

【基本問題５】 配列と分割コンパイル（ ~/Prog3/C1/make_array/ ）

以下のファイルを作成せよ．ただし、array_operations.c 内の関数は、例題８で作成したものである。これらのファイルから、array.txt を読み込んで、その内容を出力する実行ファイル array_test が作成できるように、適切に makefile を作成し、その makefile の内容を提出せよ。

C1.h

```
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>

int *random_array(int);
int count(char *);
int *read_array(char*, int);
void show_array(int*, int);
```

main.c

```
#include "C1.h"

int main(int argc, char **argv) {
    int n, *TEST;

    n = count(argv[1]);
    TEST = read_array(argv[1], n); //  読み込んだファイルを配列に格納し、先頭アドレスを TEST に代入
    show_array(TEST, n);

    free(TEST);
    return 0;
}
```

array_operations.c

```
#include "C1.h"

int count(char *filename) {
    例題 7 の count 関数と同じ
}

int *read_array(char *filename, int k) {
    例題 7 の read_array 関数と同じ
}

void show_array(int *A, int k) {
    例題 7 の show_array 関数と同じ
}
```

array.txt

```
123
456
```

```
11:  789
12:  1011
13:  1213
14:  1415
15:  1617
16:  1819
```

【コンパイル等】

- `make`

【実行】

- `./array_test array.txt`

【発展課題１】　ファイルを読み込んで処理する

入力として与えられたファイルに含まれる整数の中で，最も登場頻度の多い整数を出力するプログラムを作成せよ．例えば、以下のファイル `array.txt` が与えられた場合、答えは `3` となる。作成したプログラムを用いて `WebClass` で提示されるファイルについて，最も登場頻度の多い整数を答えよ．ただし，ファイルに含まれる整数値の大きさは `0` 以上 `1000` 未満となっている．

array.txt

```
1:   1
2:   3
3:   3
4:   2
5:   1
6:   4
7:   0
8:   3
```

【目標とする動作】

- `./occurrence array.txt` （実行ファイルの名前が `occurrence` の場合）

  `3`　　（登場頻度の多い数字を出力）